Lesson 08 - Introduction to Inverse Kinematics

Physical AI | Module 2: Robot Kinematics
Status: Core lesson complete; external Isaac Sim visual prototype tested
Learning format: Theory, mathematical derivation, Python, simulation, verification, reflection

> **Scope and evidence:** This manual documents the exercises and results reported during the lesson. It is not an independent inspection of the student's local files. Terminal output is cited as reported in the conversation; screenshots and local source files have not been included here.

1. Learning objectives

By completing this lesson, you should be able to:

1. Distinguish inverse kinematics (IK) from forward kinematics (FK).
2. Determine whether a planar target is geometrically reachable by a two-link arm.
3. Compute two IK branches, using opposite signs for the relative elbow angle.
4. Validate IK solutions by independently evaluating FK and numerical errors.
5. Transfer calculated joint configurations into Isaac Sim using visual USD geometry.
6. Explain the difference between a kinematic visualizer and a physics-driven articulated robot.

2. Concept: FK versus IK

Forward kinematics: Given link lengths and joint angles, calculate the end-effector position.

Inverse kinematics: Given link lengths and a desired end-effector position, calculate the joint angles that reach it.

For a planar two-revolute-joint arm with link lengths L_1, L_2, shoulder angle θ_1, and elbow angle θ_2 *relative to link 1*:

$$x = L_1 \cos\theta_1 + L_2 \cos(\theta_1 + \theta_2)$$

$$y = L_1 \sin\theta_1 + L_2 \sin(\theta_1 + \theta_2)$$

Angles are measured in the XY plane about the Z axis. Python's `math` trigonometric functions use radians.

3. Reachability

Let the target be (x_t, y_t), with radial distance

$$r = \sqrt{x_t^2 + y_t^2}.$$

For an ideal two-link arm with no joint limits or obstacles, a target is geometrically reachable when

$$|L_1 - L_2| \leq r \leq L_1 + L_2.$$

In the Python lesson, $L_1 = 4$ m and $L_2 = 3$ m, so the radial range is **1 to 7 m**. The following checks were tested:

Target	Distance	Expected classification
(8, 0)	8.0 m	Unreachable: too far
(3, 4)	5.0 m	Reachable
(0.5, 0)	0.5 m	Unreachable: too close

The combined IK script rejected an out-of-range target before executing `acos`. During development, uncaught `ValueError` was replaced with `SystemExit` for a readable standalone command-line message.

Limitation: Geometric reachability does not account for joint limits, collision avoidance, link thickness, or dynamics.

4. Closed-form two-link IK

First find the elbow cosine using the law of cosines:

$$c_2 = \frac{x_t^2 + y_t^2 - L_1^2 - L_2^2}{2L_1 L_2}.$$

Then there are two elbow branches:

$$\theta_{2,+} = \arccos(c_2), \qquad \theta_{2,-} = -\arccos(c_2).$$

For **each** elbow branch, recompute the shoulder angle:

$$\alpha = \text{atan2}(y_t, x_t),$$

$$\beta = \text{atan2}(L_2 \sin\theta_2,\ L_1 + L_2 \cos\theta_2),$$

$$\theta_1 = \alpha - \beta.$$

Changing only θ_2 while holding θ_1 fixed generally moves the end effector. Independent joints are not the same as independent IK solutions: the angles must satisfy a common position constraint.

Numerical robustness: Once the target has passed reachability checks, clamp c_2 into $[-1, 1]$ before `acos` to guard against floating-point drift at reach boundaries.

5. Worked example: two solutions, same target

Use $L_1 = 4$ m, $L_2 = 3$ m and target $(2.5, 4.3301270189)$ m. The target radius is approximately 5 m.

$$c_2 = \frac{5^2 - 4^2 - 3^2}{2(4)(3)} = 0.$$

Thus the elbow candidates are approximately $+90^\circ$ and -90°, and $\alpha \approx 60^\circ$.

Quantity	Positive-elbow branch	Negative-elbow branch
θ_2	$+90.00^\circ$	-90.00°
β	$+36.87^\circ$	-36.87°
θ_1	23.13°	96.87°
FK x	2.500000 m	2.500000 m
FK y	4.330127 m	4.330127 m

The student's negative-elbow combined-script output included:

```
Negative elbow angle: -90.00 degrees
Negative-elbow beta: -36.87 degrees
Negative-elbow shoulder: 96.87 degrees
Negative-elbow FK X: 2.500000 m
Negative-elbow FK Y: 4.330127 m
X error: -8.881784197001e-16 m
Y error: 0.000000000000e+00 m
Solution B: PASSED
```

The positive branch reported approximately 23.13° shoulder, $+90.00^\circ$ elbow, and matching FK coordinates. The reported errors were within the tolerance 10^{-6} m; the full final positive-branch pass/fail line was not included in that last pasted output.

A value printed as `-0.000000` is formatting of a tiny negative floating-point result, **not** a physically meaningful negative distance.

6. Python reference implementation

This standalone reference is a concise, self-contained version of the calculations developed in the lesson. It is **not claimed to be byte-for-byte identical** to the student's saved scripts.

```
import math

L1, L2 = 4.0, 3.0
x_target, y_target = 2.5, 4.3301270189
tolerance = 1e-6

r = math.hypot(x_target, y_target)
if not abs(L1 - L2) <= r <= L1 + L2:
    raise SystemExit("IK stopped: Target is geometrically unreachable")

c2 = (r*r - L1*L1 - L2*L2) / (2 * L1 * L2)
c2 = max(-1.0, min(1.0, c2))
alpha = math.atan2(y_target, x_target)

for branch, theta2 in (
    ("positive", math.acos(c2)),
    ("negative", -math.acos(c2)),
):
    beta = math.atan2(
        L2 * math.sin(theta2), L1 + L2 * math.cos(theta2)
    )
    theta1 = alpha - beta
    x_fk = L1 * math.cos(theta1) + L2 * math.cos(theta1 + theta2)
    y_fk = L1 * math.sin(theta1) + L2 * math.sin(theta1 + theta2)
    error = math.hypot(x_fk - x_target, y_fk - y_target)
    print(f"{branch}: shoulder={math.degrees(theta1):.2f} deg, "
          f"elbow={math.degrees(theta2):.2f} deg, "
          f"FK error={error:.3e} m, PASS={error < tolerance}")
```

Procedure: Save the script in the Lesson 8 folder, run with regular Python, and change the target to verify both reachable and unreachable cases. Preserve the original tested scripts separately.

7. Isaac Sim visual experiment and portfolio prototype

The Isaac Sim experiment **uses different geometry** from the first Python example:

- **Link 1:** 3 m visual cube.
- **Link 2:** 2 m visual cube.
- **Target marker:** sphere in the XY plane.

- **Scene:** `project_01_two_link_ik.usd` (the actual saved extension was `.usd`, not `.usda`).
- **Controller:** `project_01_ik_controller.py`, executed using Isaac Sim's external `python.bat`.
- **Interactive launcher:** `project_01_ik_interactive.py`, which accepts multiple targets and preserves successful generated poses.

These are independent world-space USD visual primitives, **not** physics-articulated robot joints.

Converting IK angles into visual cube transforms

For a target and one IK branch, compute

$$E = (L_1 \cos\theta_1,\ L_1 \sin\theta_1)$$

for the elbow point, and $\phi = \theta_1 + \theta_2$ for the second link's absolute rotation. Then:

$$C_1 = \tfrac{1}{2}E$$

$$C_2 = E + \tfrac{1}{2}L_2(\cos\phi, \sin\phi)$$

Cube 1 is centered at C_1 and rotated by θ_1; cube 2 is centered at C_2 and rotated by ϕ. Each cube's X extent represents its link length.

Reported Isaac Sim validation

Trial	Target (m)	Branch	Expected angles	Reported viewport result
Initial visual alignment	(3, 2)	Positive	$0°, +90°$	Tip aligned with sphere
Generated pose A	(2, 3)	Positive	$22.620°, +90°$	Tip aligned with sphere
Generated pose B	(2, 3)	Negative	$90°, -90°$	Tip aligned; alternative elbow layout
Generated pose C	(3, 2)	Positive	$0°, +90°$	Tip aligned
Unreachable case	(6, 0)	Positive	No solution	Rejected; no pose saved

The controller printed for target `(2, 3)`, positive branch:

```
Target: (2.000, 3.000)
Elbow configuration: positive
Shoulder angle: 22.620 deg
Elbow angle: 90.000 deg
FK position error: 0.000e+00 m
Generated scene: ...\project_01_ik_generated.usda
```

8. Debugging lessons and fixes

Symptom	Root cause or diagnosis	Fix used
USD stage failed to open	Controller sought `project_01_two_link_ik.usda`; saved file was `.usd`	Correct source filename
Missing prim `/World/IK_Target`	Stage traversal showed marker as `/World/Sphere`	Use actual prim path
`incompatible xformable` warnings	`XformCommonAPI` could not interpret existing transform operations	Rebuild explicit translate/rotate/scale xform op order
`xformOp:scale` float-vs-double mismatch	Existing USD scale attribute had `double3` type	Specify matching `PrecisionDouble` ops
Script Editor failed earlier	Isaac Sim Script Editor environment issue	Use standalone Isaac Sim `python.bat` instead

The resulting controller generated USD scenes that were opened and checked in the viewport.

9. Reproducibility checklist

1. Open the original `project_01_two_link_ik.usd` scene, confirm prim paths `/World/Link1`, `/World/Link2`, and `/World/Sphere`.
2. Use Isaac Sim's `python.bat` from its installation directory to run the controller.
3. Supply `X`, `Y`, and `positive` or `negative` arguments.
4. Open or reload the generated `.usda` scene in Isaac Sim; generated scenes are **not** live-updated in an already open viewport.
5. Verify end-effector/target alignment and record the FK error.
6. Use the launcher for repeated targets, including an unreachable case, and preserve individual poses.

10. Evidence to retain

Save evidence in your local Lesson 8 folder or future portfolio repository. These files still need to be captured or attached by the student; they are **not embed-**

ded in this manual:

- `positive_elbow.png` - target `(2,3)`, positive branch.
- `negative_elbow.png` - target `(2,3)`, negative branch; use same camera view.
- `unreachable_terminal.png` - target `(6,0)` rejected.
- Tested controller and interactive launcher `.py` files.
- Original `.usd` scene and a pair of generated `.usda` scenes.

11. What this lesson does NOT yet demonstrate

This visual experiment does not establish motor control, articulated dynamics, collision handling, physical joint constraints, ROS integration, trajectory planning, or live actuation. Those subjects belong in subsequent robotics lessons and larger projects.

12. End-of-lesson review

1. What changes between FK and IK inputs and outputs?
2. Why is the reachable region an annulus for unequal link lengths?
3. Why can two elbow angles reach the same point?
4. Why must the shoulder be recomputed when the elbow branch changes?
5. What does the FK verification error prove, and what does it not prove?
6. Why is a USD cube-transform visualizer different from an articulated robot?

Lesson completion criteria: A two-link IK solver supports reachability checks and both branches, validates both via FK, and demonstrates a target-reaching configuration in Isaac Sim. Those criteria were met according to the student's reported outputs and viewport observations.

Next: Proceed to Lesson 9 in accelerated mode; keep the documented lesson manual as the checkpoint artifact. Larger portfolio projects will be developed after the relevant foundational modules rather than replacing lessons.